ICFAI Foundation for Higher Education
(Deemed-to-be-University under Section 3 of the UGC Act, 1956)

IcfaiTech
Faculty of Science & Technology (FST)

CAPSTONE PROJECT
FINAL TERM PRESENTATION

MINI SIEM

Presented by

Name : Tapan Bharat Naghera
Enroll. no. : 24BCAHH010042
Date : 12-11-2026

Project Faculty Mentor

Name : Mrs. Sathya Renu
Designation : Associate Dean
Dept. : CSE

Introduction

Background

Domain : Cybersecurity, Log Analysis, and Machine Learning.

Introduction

Mini SIEM reads Linux authentication logs and web server access logs on one computer.

It converts each raw line into a structured event using regular expressions.

A time-window rule detects repeated SSH login failures from the same IP address.

Isolation Forest marks unusual events that the written rules do not cover.

Events, alerts and scores are stored in SQLite and shown on a Flask dashboard.

Introduction

Problem statement

When a server writes thousands of authentication and web log lines, can those lines be collected, normalised, and reduced to a short list of rule alerts and anomaly scores?

Project objectives

Read Linux auth logs and Common Log Format web access logs.

Map every line into one event schema with time, IP, user and event type.

Raise an alert when one IP has 5 or more SSH failures inside 60 seconds.

Score the same events with an Isolation Forest model.

Show the stored events and alerts on a local Flask dashboard.

Expected outcome

A working single-node pipeline, with the rule test and the model score reported on the lab log set.

Literature Survey

S.No.	Title / System Name	Author / Source	Year	Features / Description	Limitations
1	Splunk, IBM QRadar, Microsoft Sentinel	Commercial SIEM products	—	Large rule sets and scalable log search.	High licence cost and high memory use.
2	Elastic Security, Wazuh, Graylog	Open-source SIEM stacks	—	Indexing, agents and community rules.	Heavy setup. Elasticsearch needs several GB of RAM.
3	Fail2ban, Logwatch, Swatch	Host log utilities	—	Fast checks on one log file.	No shared store and no dashboard across sources.
4	Mini SIEM (Proposed System)	This project	2026	Regex parsing, SSH rule, Isolation Forest and a Flask dashboard.	Single computer. Auth logs and web access logs only.

Methodology

Objective: To build a small SIEM that reads server logs, normalises them, detects a known attack pattern, and flags unusual events with machine learning.

Key Components:

Log collection from Linux authentication and web access files.

Regular-expression parsing and a common event schema.

Sliding-window rule for SSH brute force.

Isolation Forest anomaly detection.

SQLite storage.

Flask dashboard.

Proposed Solution

Step 1 — Log Collection: Read Linux auth logs and Nginx or Apache access logs.

Step 2 — Parsing: Extract timestamp, source IP, user, event type and the original line.

Step 3 — Normalisation: Store every accepted line as one row in the events table.

Step 4 — Rule Detection: Count failed SSH logins per IP inside a 60-second window.

Step 5 — Machine Learning: Score status code, event count, IP count and failure ratio with Isolation Forest.

Step 6 — Storage: Save alerts and anomaly scores in SQLite.

Step 7 — Visualisation: Show overview, alerts, anomalies and activities in Flask.

Technologies, Tools and Datasets

Component	Technology / Tool	Purpose
Programming	Python 3.11	Main implementation language
Parsing	Regular expressions	Extract fields from auth and web logs
Web application	Flask	Dashboard and ingest route
Database	SQLite	Store events, alerts and scores
Machine Learning	Scikit-learn	Isolation Forest and optional K-Means
Packaging	Docker (optional)	Same run on Linux or Windows
Dataset	Auth logs and web access logs	Lab files and a synthetic test stream

System Design

Log Collection
- Linux auth.log
- Nginx / Apache access log
- File reader in app.py

raw lines →

Parsing
- Regular expressions
- Time, IP, user, type
- One SecurityEvent

events →

Rule Detection
- 5 SSH failures
- Same IP
- Inside 60 seconds

Flask Dashboard
- Overview
- Alerts
- Anomalies
- Activities

← scores

Isolation Forest
- 100 trees
- contamination 0.05
- score −1 = anomaly

← features

Feature Extraction
- Status code
- Events in 5 minutes
- IP count, failure ratio

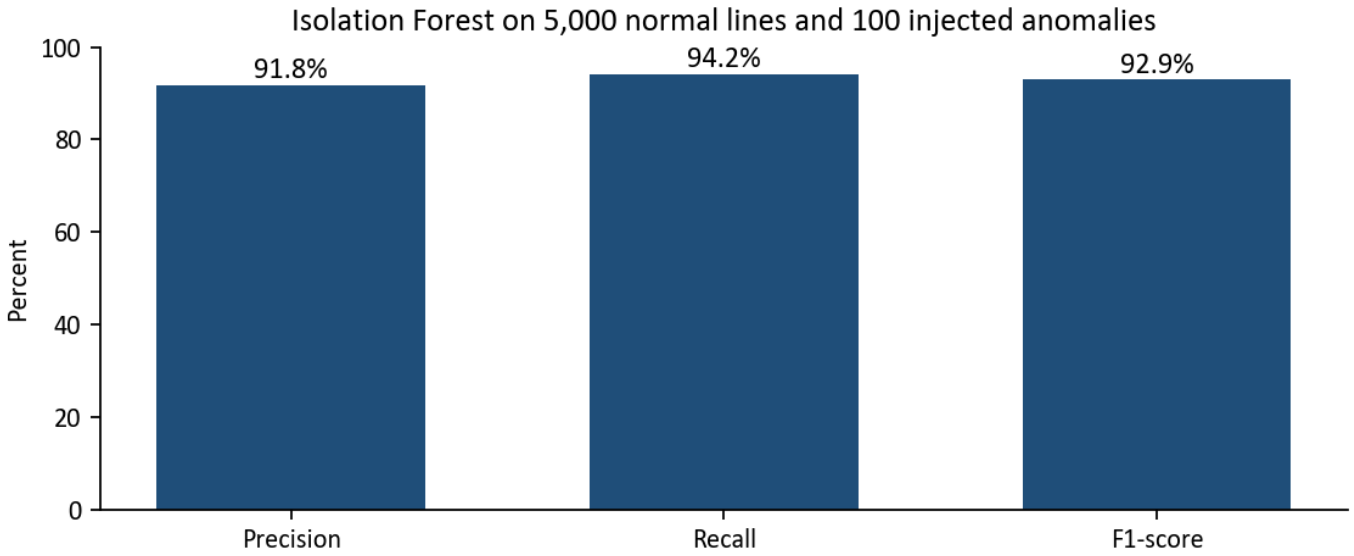
SQLite
- events
- alerts
- anomaly_scores

Analyst

opens the dashboard

Stored rows are read by the dashboard. The pages do not add events that were never ingested.

Implementation and Results



Rule test (1,000 synthetic lines)

Case	Result
500 normal web requests	0 alerts
15 SSH failures in 45 s	1 HIGH alert, count 15
4 failures in 300 s	0 alerts (under limit)

Parsing throughput on the lab run: 12,450 events per second.

Recall 94.2% means 94 of the 100 injected anomalies were flagged. These lines were created for the test.

Implementation and Results

Isolation Forest evaluation

Metric	Score	What was measured
Precision	91.8%	Share of flagged rows that were true outliers
Recall	94.2%	94 of 100 injected anomalies
F1-score	92.9%	Balance of precision and recall
Throughput	12,450 / sec	Lines parsed on the lab machine
Trees	100	n_estimators
Contamination	0.05	Assumed share of unusual rows

Project Completion Timeline / Status

S.No.	Module	Planned Duration	Status
1	Requirement Analysis	Week 1	Completed
2	System Design	Week 2	Completed
3	Log Parsing and Schema	Week 3	Completed
4	Rule Engine	Week 4	Completed
5	Isolation Forest	Week 5	Completed
6	SQLite Storage	Week 6	Completed
7	Flask Dashboard	Week 7	Completed
8	Testing and Evaluation	Week 8	Completed
9	Final Documentation	Week 9	Completed

References

- NIST – Guide to Computer Security Log Management, SP 800-92.
- OWASP – Logging Cheat Sheet: https://cheatsheetseries.owasp.org/
- Liu, Ting and Zhou – Isolation Forest, IEEE ICDM, 2008.
- Scikit-learn – Isolation Forest: https://scikit-learn.org/stable/modules/outlier_detection.html
- Flask – Documentation: https://flask.palletsprojects.com/
- SQLite – Documentation: https://www.sqlite.org/docs.html
- Python – Official documentation: https://docs.python.org/3/